Regularization

Problem: Overfitting

Large coefficients → Overfitting

Complex models may have smaller training errors but perform poorly on new data.

Solution: Penalize Large Weights

Modify the error function to punish large weights.

New Error = Old Error + Weight Penalty

L1 vs L2 Regularization

L1 (Lasso)

$$\text{Error} = \frac{1}{m}\sum_{i=1}^{n}(1-y_i)\ln(1-\hat{y}_i) + y_i\ln(\hat{y}_i) + \lambda(|w_1| + \ldots + |w_n|)$$

L2 (Ridge)

$$\text{Error} = \frac{1}{m}\sum_{i=1}^{n}(1-y_i)\ln(1-\hat{y}_i) + y_i\ln(\hat{y}_i) + \lambda(w_1^2 + \ldots + w_n^2)$$

λ (lambda): Controls penalty strength

- Large λ → More penalty → Simpler model
- Small λ → Less penalty → More complex model

When to Use Which?

L1 Regularization

- **Result:** Sparse vectors (many weights → 0)
- **Use for:** Feature selection
- **Example:** $(1, 0, 0, 3, 0)$

L2 Regularization

- **Result:** Small homogeneous weights
- **Use for:** Training models (generally better performance)
- **Example:** $(0.5, 0.3, -0.2, 0.4, 0.1)$

Why L1 Creates Sparse Vectors?

Vector (1, 0):

- L1: $|1| + |0| = 1$
- L2: $1^2 + 0^2 = 1$

Vector (0.5, 0.5):

- L1: $|0.5| + |0.5| = 1$
- L2: $0.5^2 + 0.5^2 = 0.5$

L2 prefers $(0.5, 0.5)$ over $(1, 0)$ because it produces a smaller sum of squares → encourages distributing weights evenly.

Dropout

Problem: Dominant Network Parts

During training, some parts of the network may develop very large weights and dominate the learning process, while other parts remain undertrained and contribute minimally.

Solution: Dropout

Randomly deactivate nodes during training to force all parts of the network to learn.

How It Works

During each epoch:

1. Randomly turn off some nodes (set probability, e.g., 20%)
2. Perform forward and backward propagation without those nodes
3. Other nodes must "pick up the slack" and participate more actively
4. Repeat with different random nodes each epoch

Dropout Probability

Parameter: Probability that each node gets dropped per epoch

- **p = 0.2** → Each node has 20% chance of being turned off each epoch
- Some nodes may drop more often, others less (randomness)
- Over many epochs, all nodes get similar treatment on average

Benefits

- Prevents co-adaptation of neurons
- Forces network to learn robust features
- Reduces overfitting
- Widely used and highly effective technique

Note

Dropout is only applied **during training**. At inference/prediction time, all nodes are active.

Local Minima Problem

The Challenge

Gradient descent finds the steepest descent direction and takes a step in that direction.

Problem: Getting Stuck

In complex loss landscapes (like mountain ranges), gradient descent can get trapped in **local minima**

- Algorithm reaches a point where no direction leads downward
- This local minimum may not be the **global minimum** (best solution)
- Training gets stuck at suboptimal solution

Solution: Random Restarts

Strategy

Start gradient descent from **multiple random initial positions** and run independently.

Benefits

- Increases probability of finding the global minimum
- At minimum, finds a better local minimum
- Simple yet effective approach

How It Works

1. Initialize weights randomly at different starting points
2. Run gradient descent from each starting point
3. Compare final results from all runs
4. Select the model with the best performance

This technique helps escape poor local minima and improves overall model performance.

Vanishing Gradient Problem

The Problem

Sigmoid function has very flat regions at the extremes:

- Derivative = 0 at far left or far right
- Small derivatives → tiny weight updates → slow/stuck training

Worse in Deep Networks

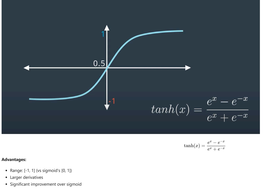
In multilayer perceptrons, the gradient is the **product of derivatives** along the path:

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial a_n} \cdot \frac{\partial a_n}{\partial a_3} \cdot \ldots \cdot \frac{\partial a_3}{\partial w}$$

- Each derivative is small (sigmoid derivatives)
- Product of small numbers → **vanishing gradient**
- Tiny weight updates → ineffective training

Solution: Better Activation Functions

1. Hyperbolic Tangent (tanh)

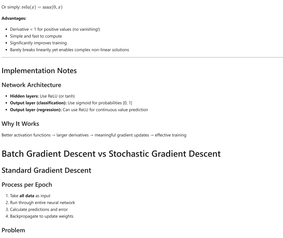


$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Advantages:

- Range: [-1, 1] (vs sigmoid's [0, 1])
- Larger derivatives
- Significant improvement over sigmoid

2. ReLU (Rectified Linear Unit)



$$relu(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Or simply: $relu(x) = \max(0, x)$

Advantages:

- Derivative = 1 for positive values (no vanishing!)
- Simple and fast to compute
- Significantly improves training
- Rarely breaks linearity yet enables complex non-linear solutions

---

Implementation Notes

Network Architecture

- **Hidden layers:** Use ReLU (or tanh)
- **Output layer (classification):** Use sigmoid for probabilities [0, 1]
- **Output layer (regression):** Can use ReLU for continuous value prediction

Why It Works

Better activation functions → larger derivatives → meaningful gradient updates → effective training

---

Batch Gradient Descent vs Stochastic Gradient Descent

Standard Gradient Descent

Process per Epoch

1. Take **all data** as input
2. Run through entire neural network
3. Calculate predictions and error
4. Backpropagate to update weights

Problem

- Huge matrix computations for large datasets
- High memory usage
- Slow training (one step per epoch with all data)

Stochastic Gradient Descent (SGD)

Key Idea

Use **small subsets of data** instead of entire dataset for each step.

Why It Works

- Small subset gives good estimate of gradient direction
- Not perfect, but fast
- Multiple approximate steps > one perfect step

Process

Example: 24 data points split into 4 batches of 6 points

1. **Batch 1:** Run 6 points → calculate error → backpropagate → update weights
2. **Batch 2:** Run 6 points → calculate error → backpropagate → update weights
3. **Batch 3:** Run 6 points → calculate error → backpropagate → update weights
4. **Batch 4:** Run 6 points → calculate error → backpropagate → update weights

Result: 4 steps taken vs 1 step with standard gradient descent

---

Comparison

Method	Steps per Epoch	Accuracy per Step	Speed	Overall Efficiency
Gradient Descent	1 (all data)	High	Slow	Low
Stochastic GD	Multiple (batches)	Lower	Fast	**High**

Key Insight

Many slightly inaccurate steps > > One accurate step

Benefits of SGD

✅ Faster training
✅ Lower memory requirements
✅ More frequent weight updates
✅ Better convergence in practice
✅ Can escape shallow local minima

Learning Rate & Momentum

Learning Rate Selection

Too Large

- Takes huge steps
- Fast initially but **overshoots minimum**
- Chaotic, unstable training
- May never converge

Too Small

- Takes tiny steps
- Slow but steadily progress
- Better chance of reaching local minimum
- Very slow training

Rule of Thumb

If your model isn't working → decrease the learning rate

Best Practice

Use **adaptive learning rates** that decrease as model approaches solution

- Keras and PyTorch provide built-in schedulers
- Start large, decay over time

Momentum

Problem

Standard gradient descent gets stuck in local minima (gradient = 0).

Solution

Use **weighted average of previous steps** to maintain direction and "power through" local minima

How It Works

Instead of just current gradient, consider:

- Previous step × ?
- Step before × β
- Step before that × β^2
- Step before that × β^3
- ...

β (beta): Momentum constant (0 < β < 1)

Effect

- Recent steps matter **most**
- Older steps matter **less** (exponential decay)
- Builds "velocity" in consistent directions
- Helps escape local minima

Benefits

✅ Escapes local minima
✅ Smooths optimization path
✅ Faster convergence
✅ Reduces oscillations

Trade-off

May overshoot global minimum slightly, but works very well in practice

---

Summary

Technique	Purpose	Key Parameter
Learning Rate	Step size control	α (alpha)
Momentum	Escape local minima	β (beta)

Combined: Adaptive learning rate + momentum = robust optimization